

10 Academy: Artificial Intelligence Mastery

Week 2 Challenge Document

Customer Experience Analytics for Fintech Apps

A Real-World Data Engineering Challenge: Scraping, Analyzing, and Visualizing Google Play
Store Reviews

Date: 13 May – 19 May 2026

Overview

Business Need

You are a **Data Analyst** at **Omega Consultancy**, a data and strategy firm advising Ethiopian banks on how to improve their mobile products and retain customers in an increasingly competitive fintech landscape.

Mobile banking adoption in Ethiopia is accelerating, and customer reviews on the Google Play Store have become one of the richest, most unfiltered signals of product quality available to product teams. Thousands of users share their experiences daily, praising fast transfers, complaining about crashes, requesting features, and abandoning apps after repeated failures. Left unanalyzed, this feedback is noise. Systematically processed, it becomes a competitive intelligence asset.

Your task is to build a rigorous analytics pipeline that transforms raw Play Store reviews into actionable insights: quantifying sentiment, identifying recurring themes, and translating them into concrete recommendations that bank product teams can act on.

Situational Overview

Your primary task is to conduct a structured, four-part analysis of user reviews for three Ethiopian banks; Commercial Bank of Ethiopia (CBE), Bank of Abyssinia (BOA), and Dashen Bank:

1. **Data Collection & Preprocessing:** Scrape and clean user reviews from the Google Play Store, producing a structured dataset ready for analysis.
2. **Sentiment & Thematic Analysis:** Apply NLP techniques to classify review sentiment and extract the key themes driving user satisfaction and dissatisfaction.
3. **Database Engineering:** Design and populate a relational PostgreSQL database to store the processed review data in a production-ready format.
4. **Insights & Recommendations:** Synthesize findings into visualizations and bank-specific product recommendations that Omega Consultancy can present to its clients.

Your final deliverable should give a product manager at any of the three banks a clear, evidence-backed picture of what their users love, what frustrates them most, and what to prioritize next.

Business Scenarios

Omega Consultancy has been engaged by three Ethiopian banks; CBE, BOA, and Dashen, to help them understand what their mobile app users are saying and why it matters. The following scenarios represent the specific consulting briefs your analysis must address.

Scenario 1: Retaining Users

CBE has a 4.2, BOA a 3.4, and Dashen a 4.1 star rating. Users complain about slow loading during transfers. Analyze whether this is a systemic issue across all three apps and suggest specific areas for product investigation.

Scenario 2: Enhancing Features

Extract desired features (e.g., fingerprint login, faster transfers, budgeting tools) through keyword and theme extraction. Recommend how each bank can stay competitive by prioritizing the right feature development.

Scenario 3: Managing Complaints

Cluster and track recurring complaints (e.g., "login error", "OTP not received") to guide customer support prioritization and inform an AI chatbot integration strategy.

Dataset Overview

You will scrape reviews from the Google Play Store for three banks' apps, collecting:

- **Review Text:** User feedback (e.g., "Love the UI, but it crashes often").
- **Rating:** 1–5 stars.
- **Date:** Posting date (YYYY-MM-DD).
- **Bank / App Name:** e.g., "Commercial Bank of Ethiopia Mobile".
- **Source:** Google Play.

Minimum: 400 reviews per bank (1,200 total). If scraping is limited by rate limits or availability, document the issue and supplement with a broader date range.

Learning Outcomes

Skills:

- **Web Scraping:** Using [google-play-scraper](#) to collect and preprocess unstructured user-generated text data.
- **NLP & Sentiment Analysis:** Applying transformer-based and lexicon-based tools to classify sentiment at scale.
- **Thematic Analysis:** Extracting keywords and clustering them into actionable themes using TF-IDF and spaCy.
- **Database Engineering:** Designing and populating a relational PostgreSQL schema using Python.
- **Data Visualization:** Creating clear, stakeholder-ready plots that communicate findings rather than raw data.
- **Unit Testing:** Writing tests to validate data pipeline reliability and correctness.

Knowledge:

- Understanding how user-generated text data (reviews, ratings) reflects underlying product quality and user needs.
- Familiarity with the trade-offs between lexicon-based sentiment tools (VADER, TextBlob) and fine-tuned transformer models (DistilBERT).
- Knowing how to design a relational schema that reflects real-world entities and relationships.
- Understanding data quality challenges specific to scraped data: duplicates, missing fields, inconsistent date formats.

Behaviors:

- **Proactivity:** Self-directed learning, consulting documentation, and sharing useful references with peers.
- **Adaptability:** Handling real-world messiness — rate limits, missing data, ambiguous reviews — with pragmatic solutions.
- **Rigor:** Committing work regularly with descriptive messages, maintaining well-documented and modular code.

Communication:

- Writing a concise, publication-style final report suitable for a Medium Blog post.
- Framing insights in business terms — not just statistical findings, but what they mean for a product manager or bank executive.

Team

Facilitators:

- Kerod
- Mahbubah
- Feven

Key Dates

- **Challenge Introduction** – 10:30 AM UTC on Wednesday, 13 May 2026.
- **Interim Submission** – 8:00 PM UTC on Sunday, 17 May 2026.
- **Final Submission** – 8:00 PM UTC on Tuesday, 19 May 2026.

Communication & Support

- **Slack channel:** #all-week2
- **Office hours:** Mon–Fri, 08:00–15:00 UTC

Tasks

Recommended Project Structure

```

None
fintech-review-analytics/
├── .vscode/
│   └── settings.json
├── .github/
│   └── workflows/
│       └── unittests.yml
├── .gitignore
├── requirements.txt
├── README.md
├── data/
│   └── raw/
├── notebooks/
│   ├── __init__.py
│   └── README.md
├── src/
│   └── __init__.py
├── tests/
│   └── __init__.py
└── scripts/
    ├── __init__.py
    └── README.md
  
```

Task 1: Data Collection and Preprocessing

Objective: Scrape reviews from the Google Play Store, preprocess them into a clean, analysis-ready dataset, and manage all code via GitHub with proper version control hygiene.

Instructions:

1) Git Setup

- Create a GitHub repository named `fintech-review-analytics`.
- Include `.gitignore` (exclude `data/`, `*.csv`, `*.db`) and `requirements.txt`.
- Configure a CI/CD workflow using GitHub Actions (`.github/workflows/unittests.yml`) that runs `pip install -r requirements.txt` on every push to `main`.
- Create a `task-1` branch. Commit frequently with descriptive messages following [Conventional Commits](#).

2) Web Scraping

- Use `google-play-scraper` to collect reviews, ratings, dates, and app names for all three banks.

- b) Target a minimum of 400+ reviews per bank (1,200 total). If the library returns fewer, expand the date range or document the limitation clearly.
 - c) Collect the following fields: review text, rating (1–5), review date, bank / app name, source ("Google Play").
- 3) **Preprocessing**
 - a) Remove duplicate reviews (id).
 - b) Handle missing values: drop rows missing review text or rating; document counts.
 - c) Normalize dates to YYYY-MM-DD format.
 - d) Save the cleaned dataset as a CSV with columns: `review`, `rating`, `date`, `bank`, `source`.
 - e) Ensure the CSV is listed in `.gitignore` and never committed to GitHub.
- 4) **README**
 - a) Document the scraping methodology, date range used, and any limitations encountered.

Key Performance Indicators (KPIs):

- 1,200+ reviews collected with <5% missing data.
- Clean CSV dataset with the required five columns.
- CI/CD workflow passes on push to `main`.
- Organized Git repository with clear, conventional commits.

Minimum Essential To Do:

- Scrape at least 400 reviews per bank (1,200 total).
- Commit a preprocessing script with inline documentation.
- Update `README.md` with scraping methodology.

Task 2: Sentiment and Thematic Analysis

Objective: Quantify review sentiment and identify recurring themes to uncover satisfaction drivers and pain points for each bank.

Instructions:

- 1) **Sentiment Analysis**
 - a) Use [distilbert-base-uncased-finetuned-sst-2-english](#) to classify each review as positive, negative, or neutral and produce a confidence score.
 - b) Alternatively, start with VADER or TextBlob and compare results with the transformer model if time permits. Document your tool selection rationale briefly.

- c) Aggregate sentiment scores by bank and by star rating (e.g., mean sentiment for 1-star reviews vs. 5-star reviews).
- 2) **Thematic Analysis**
 - a) A theme is a recurring concept or category in user feedback that maps to a business-relevant issue (e.g., "Account Access Issues", "Transaction Performance", "UI & Design", "Customer Support", "Feature Requests").
 - b) Extract significant keywords and n-grams using TF-IDF or spaCy (e.g., "login error", "slow transfer", "good UI").
 - c) Group related keywords into 3–5 overarching themes per bank. Document your grouping logic in a markdown cell.
 - d) Optionally, use topic modeling (LDA or NMF) or zero-shot classification to assist with theme discovery.
- 3) **Pipeline**
 - a) Write a modular script that handles: tokenization, stop-word removal, and optional lemmatization using spaCy or NLTK.
 - b) Save results as a CSV with columns: `review_id`, `review_text`, `sentiment_label`, `sentiment_score`, `identified_theme`.
- 4) **Git**
 - a) Create a `task-2` branch. Commit scripts and notebooks. Merge into `main` via a Pull Request.

Key Performance Indicators (KPIs):

- Sentiment scores assigned to 90%+ of reviews.
- 3+ distinct themes per bank, each supported by keyword examples.
- Modular, reusable pipeline code with documented logic.

Minimum Essential To Do:

- Sentiment scores for at least 400 reviews.
- 2 themes per bank identified through keyword analysis.
- Commit analysis script to the `task-2` branch.

Task 3: Store Cleaned Data in PostgreSQL

Objective: Design and implement a relational database schema in PostgreSQL to persistently store the cleaned and processed review data, simulating a real-world data engineering workflow.

Instructions:

1. **Database Setup**
 - a. Install PostgreSQL on your system.

- b. Create a database named `bank_reviews`.
2. **Schema Design**
 - a. **Banks Table:** Stores metadata about the banks.
 - Suggested columns: `bank_id` (PRIMARY KEY), `bank_name`, `app_name`
 - b. **Reviews Table:** Stores the scraped and processed review data.
 - Suggested columns: `review_id` (PRIMARY KEY), `bank_id` (FOREIGN KEY), `review_text`, `rating`, `review_date`, `sentiment_label`, `sentiment_score`, `identified_theme`, `source`
3. **Data Insertion**
 - a. Use Python (`psycopg2` or `SQLAlchemy`) to insert the cleaned review data into the database.
 - b. Write SQL queries to verify data integrity: count reviews per bank, compute average rating per bank, and check for nulls in key columns.
4. **Documentation**
 - a. Commit a SQL schema file (`.sql` dump or `schema.sql`) and document the setup process in [README.md](#).
5. **Git**
 - a. Create a `task-3` branch. Merge into `main` via a Pull Request.

Key Performance Indicators (KPIs):

- Working database connection and insert script.
- Tables populated with >1,000 review entries.
- SQL schema file committed to GitHub.
- Verification queries executed and results documented.

Minimum Essential To Do:

- PostgreSQL database created with both tables (banks and reviews).
- Python script that successfully inserts at least 400 reviews.
- Schema documented in [README.md](#).

Task 4: Insights and Recommendations

Objective: Synthesize the sentiment and thematic analysis into business-actionable insights, supported by clear visualizations and concrete product recommendations.

Instructions:

1. **Insights**
 - a. Identify at least 2 satisfaction drivers (e.g., fast navigation, reliable transfers) and 2 pain points (e.g., frequent crashes, failed OTP) per bank, with supporting evidence from the data.

- b. Compare banks across key dimensions: overall sentiment distribution, average rating, and dominant themes.
 - c. Propose at least 2 concrete product or support improvements per bank, grounded in the data.
2. **Visualization**
 - a. Create 3–5 plots using Matplotlib or Seaborn:
 - Sentiment distribution by bank (e.g., stacked bar chart).
 - Rating distribution per bank (e.g., histograms or boxplots).
 - Top keywords or theme frequency per bank (e.g., horizontal bar charts or word clouds).
 - Sentiment trend over time (if date range is sufficient).
3. **Git**
 - a. Create a **task-4** branch. Commit visualization scripts and the final report. Merge into **main** via a Pull Request.

Key Performance Indicators (KPIs):

- 2+ drivers and 2+ pain points per bank, each supported by evidence from the analysis.
- Clear, labeled, and well-titled visualizations.
- Concrete, prioritized product recommendations per bank.

Minimum Essential To Do:

- 1 driver and 1 pain point per bank.
- 2 plots (e.g., sentiment bar chart and keyword frequency chart).
- Final report of up to 10 pages in Medium blog style (PDF format).

Due Date (Submission)

Sunday: (18 May, 2026 8:00 PM) (UTC)

What to Submit:

- **GitHub Link:** Main branch with **task-1** merged and partial progress on **task-2**.
- **Interim Report** (maximum 5 pages) covering:
 - Scraping methodology and data quality summary.
 - Early sentiment findings with at least 1 visualization.
 - Any blockers encountered and your plan for the final submission.

Tuesday: (20 May, 2026 8:00 PM) (UTC)

What to Submit:

- **GitHub Link:** Main branch with all tasks completed, clean and well-documented code.
- **Final Report** (maximum 15 pages, maximum 15 plots), written in Medium blog style (PDF format):
 - Executive summary of the challenge and your approach.
 - Data collection methodology and quality assessment.
 - Sentiment analysis methodology, tool selection rationale, and results.
 - Thematic analysis findings with keyword and theme summaries.
 - Database design overview.
 - Insights, visualizations, and bank-specific recommendations.
 - Ethical considerations and limitations.
 - Suggested next steps.

Other Considerations:

- **Documentation:** Comment your code and keep **README.md** up to date.
- **Collaboration:** Use GitHub Issues for task coordination and discussion.
- **Communication:** Regular check-ins, Q&A sessions, and a supportive community atmosphere.
- **Flexibility:** If scraping fails or is rate-limited, document the issue clearly and use an extended date range or a backup dataset.
- **Professionalism:** Emphasise work ethics and professional behavior.
- **Time Management:** Stress the importance of punctuality and managing time effectively.

Tutorials Schedule

In the following, the color **purple** indicates morning sessions, and non-purple indicates afternoon sessions.

- Day 1 (Wednesday 13 May 2026):
 - Challenge Intro (Kerod)
 - Scraping Basics, Data Cleaning and Preprocessing Strategy (Mahbubah)
- Day 2 (Thursday 14 May 2026):
 - Customer Experience in Fintech, Sentiment Analysis, and Keyword Extraction (Feven)
 - Database Fundamentals & PostgreSQL Setup (Mahbubah)
- Day 3 (Friday 15 May 2026):
 - Error Handling and Unit Testing (Feven)
 - Insight Communication and Plotting for Stakeholders (Mahbubah)
- Day 4 (Monday 18 May 2026):
 - Q&A
- Day 5 (Tuesday 19 May 2026):
 - Q&A

Feedback

You will receive comments/feedback in addition to a grade.

References

Web Scraping (Google Play Store)

- [PyPI Page](#)
- [GitHub Repository](#)

Data Handling & Preprocessing

- [10 Minutes to pandas](#)
- [Working with Text Data – pandas](#)
- [Handling Missing Data – pandas](#)

Sentiment Analysis & NLP

- [Hugging Face Transformers – Sentiment Analysis Pipeline:](#)
- [Model Card \(distilbert-base-uncased-finetuned-sst-2-english\)](#)
- [VADER Sentiment – GitHub](#)
- [TextBlob Quickstart](#)

Thematic Analysis (Keyword Extraction & Topic Modeling)

- [spaCy 101](#)
- [Scikit-learn TF-IDF \(TfidfVectorizer\)](#)
- [Scikit-learn LDA \(LatentDirichletAllocation\)](#)
- [Hugging Face Zero-Shot Classification](#)
- [Sentence Transformers Quickstart](#)

Database (PostgreSQL)

- [PostgreSQL Getting Started](#)
- [PostgreSQL Downloads](#)
- [psycopg2 Documentation:](#)
- [SQLAlchemy PostgreSQL Dialect](#)
- [pgAdmin \(GUI Tool\)](#)
- [W3Schools SQL Tutorial](#)
- [PostgreSQL Tutorial](#)

Unit Testing

- [pytest Getting Started](#)
- [Python unittest – Basic Example](#)
- [A Gentle Introduction to Unit Testing in Python – Machine Learning Mastery](#)

Version Control – Git

- [What is version control – Atlassian](#)
- [Learn Git Branching – Interactive](#)
- [Which files to not track – Atlassian](#)
- [Conventional Commits](#)

CI/CD

- [What is Continuous Integration – Atlassian](#)
- [DevOps Pipeline – Atlassian](#)
- [Setting up a CI/CD Pipeline on GitHub](#)